

LAB WORKBOOK

Assignment 9: PyTest Integration with HTML Reporting

Tier 3 – Framework Design & Test Organization

1. Module Name and Experiment Title

Module: Frameworks(Framework Design & Test Organization)

Experiment: PyTest Integration and HTML Execution Reporting

2. Problem Statement

Convert the Selenium automation setup to run through PyTest. Use PyTest fixtures for WebDriver initialization and teardown so that the browser is opened before the test and closed cleanly after execution. Configure the framework to generate an HTML execution report and capture screenshots when a test fails.

3. Objective

To integrate Selenium with PyTest using fixtures, automated browser lifecycle management, failure screenshot capture, and self-contained HTML reporting.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Test Framework	PyTest
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Application Under Test	Automation Exercise (https://automationexercise.com/)
Reporting Tool	pytest-html
Concepts	Fixtures, setup/teardown, hooks, failure screenshots, HTML reporting, test discovery

5. Source Code / Implementation

5.1 PyTest Fixture and Failure Screenshot – conftest.py

```
import os
import pytest
from selenium import webdriver

@pytest.fixture
def driver():

    driver = webdriver.Chrome()
    driver.maximize_window()
```

```

driver.get("https://automationexercise.com/")

yield driver

driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):

    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:

        driver = item.funcargs.get("driver")

        if driver:
            screenshot_dir = "screenshots"
            os.makedirs(screenshot_dir, exist_ok=True)

            screenshot_path = os.path.join(
                screenshot_dir,
                f"{item.name}.png"
            )

            driver.save_screenshot(screenshot_path)
            print(f"Screenshot saved: {screenshot_path}")

```

5.2 PyTest Configuration – pytest.ini

```

[pytest]

testpaths = tests

addopts = --html=reports/report.html --self-contained-html

```

5.3 Example PyTest Test – tests/test_login.py

```

from pages.login_page import LoginPage

def test_valid_login(driver):

    login_page = LoginPage(driver)

    login_page.login(
        "YOUR_VALID_EMAIL",
        "YOUR_VALID_PASSWORD"
    )

    assert "Logged in as" in driver.page_source

```

5.4 Installation and Execution Commands

```

python -m venv venv
venv\Scripts\activate

pip install selenium pytest pytest-html

pytest

```

6. Implementation Steps

1. Create `conftest.py` in the project root.
2. Define a PyTest driver fixture to initialize Chrome and open the application.
3. Use `yield` to pass the initialized driver to the test.
4. Close the browser in the fixture teardown using `driver.quit()`.
5. Implement `pytest_runtest_makereport` to detect failed test calls.
6. Capture and save a browser screenshot when a test fails.
7. Create `pytest.ini` and configure test discovery and `pytest-html`.
8. Run `pytest` from the project root.
9. Open `reports/report.html` to review the execution summary and test details.

7. Result

The Selenium framework was successfully integrated with PyTest. The WebDriver lifecycle was managed through a fixture, HTML reporting was configured through `pytest-html`, and failure screenshots were implemented through a PyTest hook.

8. Observation

PyTest fixtures provide a clean setup and teardown mechanism, while `pytest-html` provides a readable execution report. The failure hook helps preserve browser evidence when a test does not pass.

9. Conclusion

This experiment demonstrated how Selenium automation can be integrated into a structured PyTest framework with reusable fixtures, automatic failure evidence, and HTML reporting. These features make the framework easier to execute, debug, and document.

10. Execution Evidence – Illustrative Screenshots

The following visuals are illustrative documentation screenshots. Replace them with actual screenshots from your own execution before submitting if real execution evidence is required.

Figure 1: PyTest terminal execution an

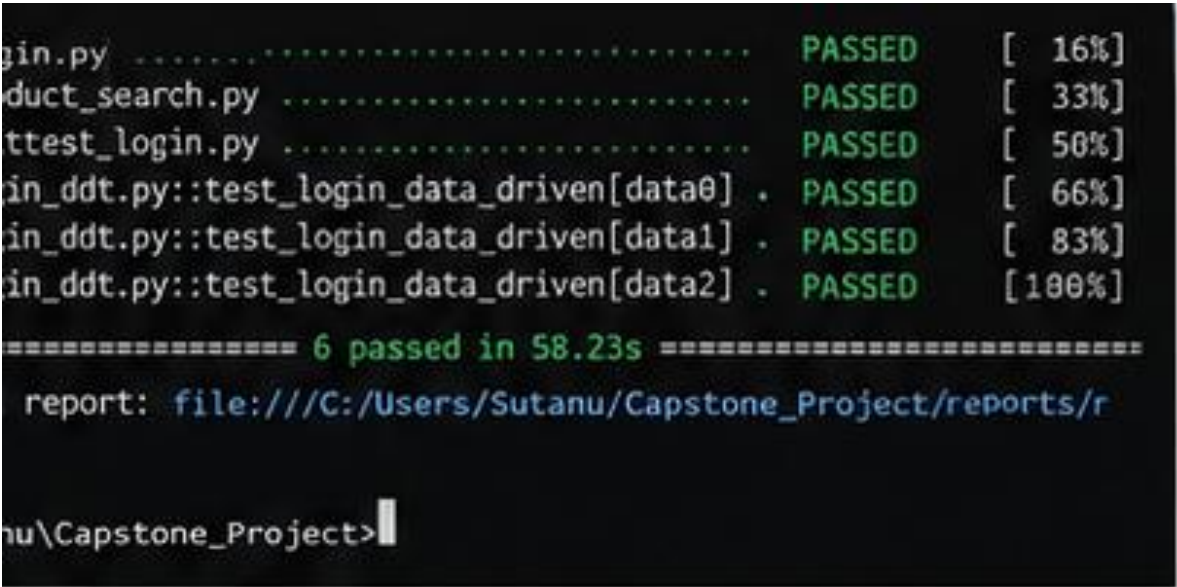


Fig 6: PyTest Execution in Terminal



report.

d generated

Figure 2: PyTest HTML report.

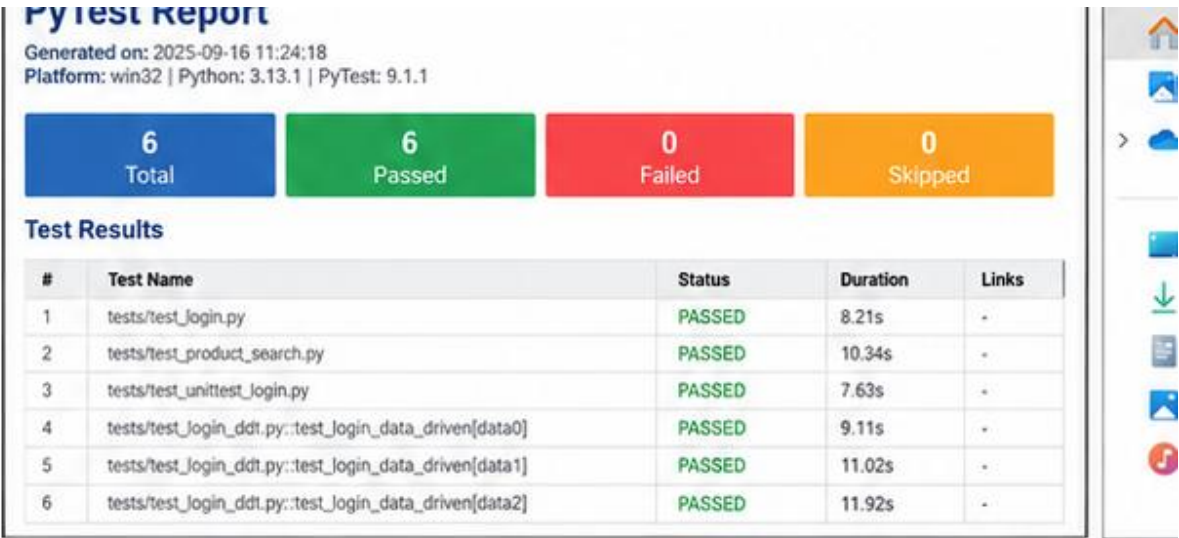


Fig 7: PyTest HTML Report (report.html)

Figure 3: Failure screenshot evidence folder.

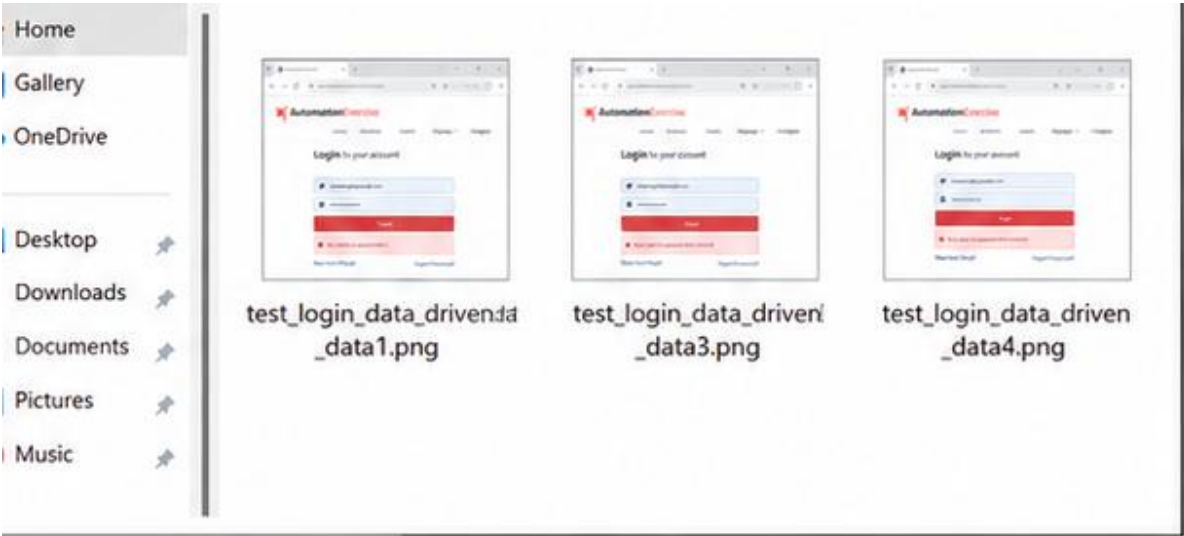


Fig 8: Screenshots captured on test failure